Smart Door Lock System

A Project Report for Industrial Training and Internship

submitted by

INDRAJIT BHOWMICK

In the partial fulfillment of the award of the degree of

B.Tech(Bachelor of Technology)

in the

Electronics And Communication Engineering

Of

Dr. Sudhir Chandra Sur Institute of Technology And Sports Complex

At

Ardent Computech Pvt. Ltd.

ARDENT®
GROUP OF COMPANIES
INTERNSHIP | DEVELOPMENT | PROJECT
An ISO 9001 : 2015 Certified Company

ARDENT COMPUTECH PVT. LTD.
High-End Technology Training and Project

Ardent Computech Pvt. Ltd.
Drives you to the Industry
Module 132, SDF Building, Sector V, Salt Lake, Pin - 700091
www.ardentcollaborations.com

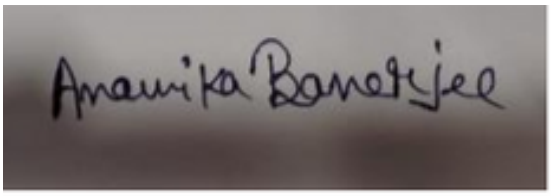
ISO 9001:2015 CERTIFIED

MSME

N·E·A·T
National Educational Alliance for Technology

CERTIFICATE FROM SUPERVISOR

This is to certify that **"Indrajit Bhowmick, 25500322044**" have completed the project titled "Smart Door Lock System" under my supervision during the period from "12th August 2024" to "24th August 2024" which is in partial fulfillment of requirements for the award of the **B.Tech** degree and submitted to the Department of "**Electronics and Communication Engineering**" of **"Dr. Sudhir Chandra Sur Institute of Technology and Sports Complex"**.



Signature of the Supervisor Date: 24/08/2024

Name of the Project Supervisor: Ms. Anamika Banerjee

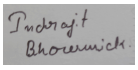
Ardent Computech Pvt. Ltd.
Drives you to the Industry
Module 132, SDF Building, Sector V, Salt Lake, Pin - 700091
www.ardentcollaborations.com

BONAFIDE CERTIFICATE

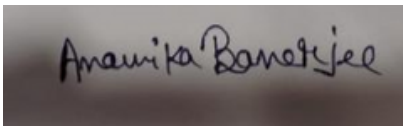
Certified that this project work was carried out under my supervision

"Smart Door Lock System" is the bonafide work of

Name of the student: Indrajit Bhowmick



Signature:



SIGNATURE

Name : Ms.Anamika Banerjee

PROJECT MENTOR

SIGNATURE

Name: Anamika Baneerjee

EXAMINERS

Ardent Original Seal

ACKNOWLEDGEMENT

The achievement that is associated with the successful completion of any task would be incomplete without mentioning the names of those people whose endless cooperation made it possible. Their constant guidance and encouragement made all our efforts successful.

We take this opportunity to express our deep gratitude towards our project mentor, ***Ms.Anamika Banerjee*** for giving such valuable suggestions, guidance and encouragement during the development of this project work.

Last but not the least we are grateful to all the faculty members of **Ardent Computech Pvt. Ltd.** for their support.

Title of the Project: – ***Smart Door Lock System Using Raspberry Pi Pico and Blynk IoT***

Name of the Guide: – ***Ms. Anamika Banerjee***

Date of Commencement: – ***12th August 2024***

Date of Completion: – ***20th August 2024***

Date of Submission: – ***24th August 2024***

Project Members: –

S.No	Member's Name
1.	Debjit Kar
2.	Joshita Bhattacharyya
3.	Rohan Roy
4.	Dhrubajyoti Maity
5.	Sayan Pradhan
6.	Tuhin Paul
7.	Swapnaneil Pal
8.	Jayee Majumdar
9.	Suchandra Bharati
10.	Indrajit Bhowmick
11.	Ananya Ghosh
12.	Arunabho Garai
13.	Debanjali De
14.	Biswayan Bhowmick
15.	Snay Rai

Table of Contents

S.No.	Particulars	Page Number
1.	Introduction	7-8
2.	What is Smart Door Lock System ?	9-10
3.	Components Required for Smart Door Lock System	11-12
4.	Description of the Components a. Raspberry Pi Pico b. Micro Servo Motor c. Photoresistor (LDR) sensor d. USB Type-B Cable e. Connecting wires(Jumper wires) • Female to Male or Male to Female • Female to Female f. Cardboard and Glue	13-26
5.	Code And Output	27-32
6.	Detailed Explanation of Smart Door Lock System using Wifi (Local IOT)	33-34
7.	Detailed Explanation of Smart Door Lock System using Blynk IOT (Gobal IOT)	35-37
8.	Circuit Diagram	38
9.	Future Scope	39-40
10.	Conclusion	40

Introduction

The **Internet of Things (IoT)** refers to a network of physical objects or "things" embedded with sensors, software, and other technologies to connect and exchange data with other devices and systems over the internet. These "things" can range from everyday household items like smart thermostats and wearable fitness trackers to industrial machines and even entire cities equipped with smart infrastructure. IoT is a rapidly growing field with the potential to transform many aspects of everyday life, industry, and society.

Key Components of IoT:

- **Devices/Things:** These are the physical objects that gather and exchange data. They can include sensors, actuators, and other embedded systems.

- **Connectivity:** IoT devices need to communicate with each other, and they do so through various forms of connectivity such as Wi-Fi, Bluetooth, Zigbee, cellular networks, and more.

- **Data Processing:** The data collected by IoT devices is often processed locally or sent to the cloud for further analysis. This can involve simple data filtering or complex data analysis using AI and machine learning.

- **User Interface:** The results of the data analysis are then made available to the end-user through interfaces like mobile apps, dashboards, or automated systems.

- **Security:** Ensuring the safety and privacy of data is a crucial aspect of IoT. This includes securing the communication channels, devices, and the cloud infrastructure used.

Applications of IoT:

- **Smart Homes:** Devices like smart thermostats, lighting systems, and home security systems can be controlled remotely, leading to more efficient energy use and increased security.

- **Industrial IoT (IIoT):** In manufacturing, IoT is used to monitor equipment, predict failures, and optimize production processes.

- **Healthcare:** Wearable devices can monitor vital signs and send alerts in case of emergencies, improving patient care and enabling remote health monitoring.

- **Smart Cities:** IoT technologies help manage urban infrastructure, such as traffic flow, energy consumption, and waste management, making cities more efficient and livable.

- **Agriculture:** IoT is used to monitor soil moisture, weather conditions, and crop health, leading to more efficient farming practices and increased crop yields.

Challenges of IoT:

- **Interoperability:** Ensuring that devices from different manufacturers can work together smoothly.
- **Security:** Protecting data from cyber threats is a major concern in IoT ecosystems.
- **Data Management:** Handling the large volumes of data generated by IoT devices efficiently and effectively.

What is a Smart Door Lock System

A **smart door lock system** using IoT is an advanced locking mechanism that utilizes internet connectivity to provide enhanced control, monitoring, and security for door access. This system allows users to lock and unlock doors remotely, monitor access, and integrate with other smart home devices, all through a connected network. A smart door lock system using IoT leverages

internet connectivity to provide enhanced control, monitoring, and security features for door access. This system is part of the broader ecosystem of smart home devices, enabling remote management and integration with other smart home technologies.

Key Components:

- **IoT-Enabled Hardware:**
  - **Smart Lock Device:** The door lock itself is equipped with electronic components such as sensors, a motorized deadbolt, and wireless communication modules (e.g., Wi-Fi, Bluetooth, or Zigbee).
  - **Sensors:** These can detect door status (open/closed) and lock status (locked/unlocked), providing real-time feedback.

- **Connectivity:** The lock connects to the internet via Wi-Fi or through a hub (using Bluetooth or Zigbee), allowing for remote control and real-time monitoring from anywhere with an internet connection.

- **Cloud Integration:** Data from the lock, such as access logs or status updates, is sent to cloud servers. This allows users to access and manage their lock via a smartphone app or web dashboard.

- **User Interface:** A mobile app or web interface where users can control the lock, view access history, and receive notifications. This interface often supports multiple access methods, such as PIN codes, fingerprint recognition, or smartphone proximity.

Applications and Benefits:

- **Residential Use:** Homeowners can enhance their security and convenience by replacing traditional locks with smart locks that offer keyless entry, remote control, and integration with other smart home devices.

- **Commercial Use:** Businesses can manage access for employees, contractors, and visitors with detailed control over who can enter specific areas and when, along with real-time monitoring.

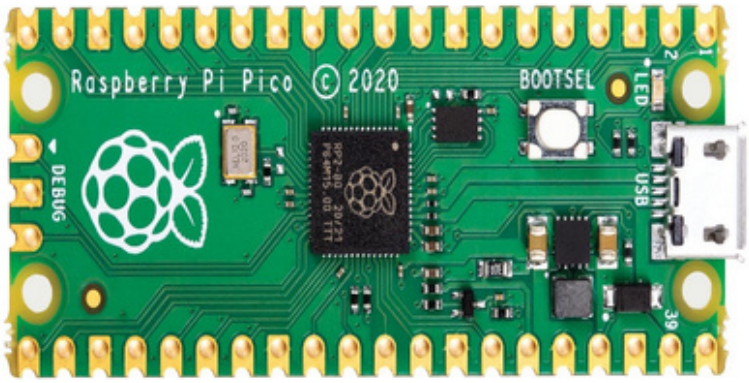
- **Vacation Rentals:** Property owners can easily manage access for guests without needing to hand over physical keys, providing a seamless check-in experience.

Challenges:

- **Security Risks:** As an IoT device, it can be vulnerable to hacking if not properly secured.
- **Reliability:** The system depends on internet connectivity and power, meaning it could fail during outages.
- **Battery Life:** Regular maintenance is needed to ensure the batteries in the lock don't die unexpectedly.

Components Required for Smart Door Lock System

1. *Raspberry Pi Pico*



1.

2. *Micro Servo Motor*

1.

2. *PIR(Passive Infrared) sensor*

1. *USB Type-B Cable*

1. *Connecting Wires*

- ***Female to Male or Male to Female***

-
- ***Female to Female***

-

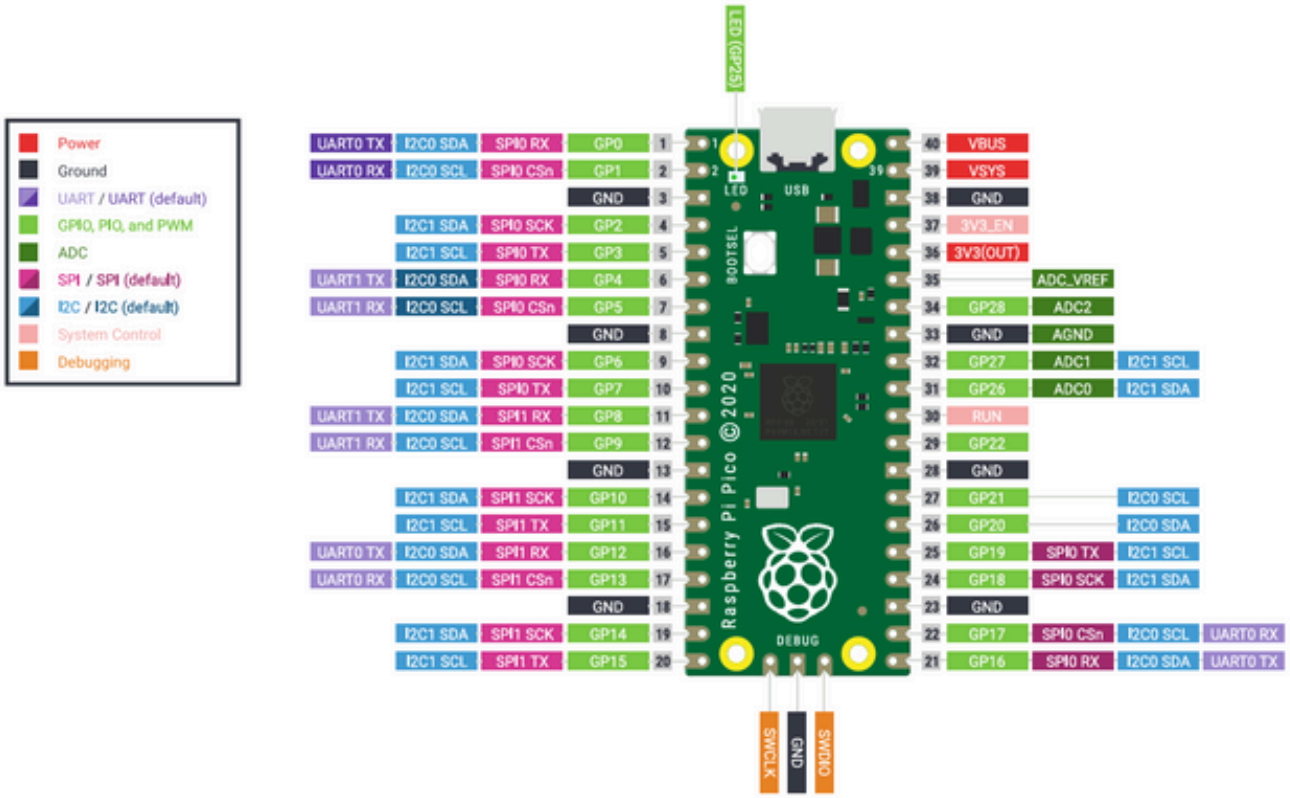
1. ***Cardboard and Glue for Door***

Description of the Components

1. *Raspberry Pi Pico*

Raspberry Pi Pico, an RP2040 microcontroller board with dual-core ARM Cortex-M0+ processor, 264k of internal RAM, and flexible Programmable I/O (PIO) feature.

The **Raspberry Pi Pico** is a small, low-cost microcontroller board developed by the Raspberry Pi Foundation. It is based on the RP2040 microcontroller chip, designed specifically for the Pico. The board is well-suited for a wide range of embedded applications and DIY electronics projects, especially those that require simple control, sensors, and automation.


Power
Ground
UART / UART (default)
GPIO, PIO, and PWM
ADC
SPI / SPI (default)
I2C / I2C (default)
System Control
Debugging
LED (GP25)
UART0 TX I2C0 SDA SPI0 RX GP0 1
UART0 RX I2C0 SCL SPI0 CSn GP1 2
GND 3
I2C1 SDA SPI0 SCK GP2 4
I2C1 SCL SPI0 TX GP3 5
UART1 TX I2C0 SDA SPI0 RX GP4 6
UART1 RX I2C0 SCL SPI0 CSn GP5 7
GND 8
I2C1 SDA SPI0 SCK GP6 9
I2C1 SCL SPI0 TX GP7 10
UART1 TX I2C0 SDA SPI1 RX GP8 11
UART1 RX I2C0 SCL SPI1 CSn GP9 12
GND 13
I2C1 SDA SPI1 SCK GP10 14
I2C1 SCL SPI1 TX GP11 15
UART0 TX I2C0 SDA SPI1 RX GP12 16
UART0 RX I2C0 SCL SPI1 CSn GP13 17
GND 18
I2C1 SDA SPI1 SCK GP14 19
I2C1 SCL SPI1 TX GP15 20
40 VBUS
39 VSYS
38 GND
37 3V3_EN
36 3V3(OUT)
35 ADC_VREF
34 GP28 ADC2
33 GND AGND
32 GP27 ADC1 I2C1 SCL
31 GP26 ADC0 I2C1 SDA
30 RUN
29 GP22
28 GND
27 GP21 I2C0 SCL
26 GP20 I2C0 SDA
25 GP19 SPI0 TX I2C1 SCL
24 GP18 SPI0 SCK I2C1 SDA
23 GND
22 GP17 SPI0 CSn I2C0 SCL UART0 RX
21 GP16 SPI0 RX I2C0 SDA UART0 TX
LED
USB
BOOTSEL
Raspberry Pi Pico ©2020
DEBUG
SWCLK
GND
SWDIO

Pin names

Pins GP0 to GP22 are digital GPIO pins. Pins GP26, GP27, and GP28 are digital GPIO pins with analog input function.

Name	Description	Analog input channel
GP0 ... GP22	Digital GPIO pins (0 to 22)	
GP26	Digital GPIO pin 26	0
GP27	Digital GPIO pin 27	1
GP28	Digital GPIO pin 28	2
GND.1 ... GND.8	Ground pins *	
VSYS, VBUS, 3V3	Positive power supply	
TP4 †	Digital GPIO pin 23	
TP5 †	Digital GPIO pin 25 + LED	

* The physical pin numbers of the ground pins are 3, 8, 13, 18, 23, 28, 33, and 38.
† These pins do not appear in the visual diagram editor, but you can use them in your diagram.json file.

Pins 3V3_EN / RUN / ADC_VREF are not available in the simulation and are therefore omitted from the table.

Key Features of Raspberry Pi Pico:

1. RP2040 Microcontroller:

- Processor: Dual-core ARM Cortex-M0+ processor running at up to 133 MHz.
- Memory: 264KB of SRAM.
- Flash Storage: 2MB of onboard QSPI flash memory for storing code and data.

2. Connectivity and I/O:

- GPIO Pins: 26 multifunctional GPIO pins, supporting digital I/O, PWM, ADC (analog-to-digital conversion), I2C, SPI, UART, and more.
- ADC: 3 analog inputs (ADC) with 12-bit resolution.

- USB: Micro USB 1.1 port for power, data transfer, and programming. It can also be used for USB-based communication.

3. Programming:

- Languages Supported: Can be programmed using MicroPython, C, or C++.

- IDE Support: Works with popular IDEs such as Thonny (for MicroPython) and the Arduino IDE (using C/C++).

4. Power:

- Power Input: Powered via the micro-USB port (5V), or via external supply through the VSYS pin (1.8V to 5.5V).

- Low Power: The RP2040 chip is designed for low-power operation, making it suitable for battery-powered applications.

5. Size and Form Factor:

- Compact Design: The Pico is very small, measuring just 51mm x 21mm, making it easy to integrate into projects where space is a concern.

6. Peripherals:

- Timers: 8 Programmable I/O (PIO) state machines for customizable peripheral interfacing.

- Internal Temperature Sensor: Provides a basic temperature reading of the microcontroller.

Applications and Use Cases:

- **DIY Electronics Projects:** Ideal for hobbyist projects involving sensors, actuators, and simple automation.
- **Prototyping:** Used to quickly prototype ideas before deploying them on more complex systems.
- **Education:** Great for learning about microcontrollers, programming, and embedded systems.
- **IoT Devices:** Can be used in simple IoT applications, such as environmental monitoring, home automation, and smart devices.

Example Projects with Raspberry Pi Pico:

- **Temperature and Humidity Monitor:** Using sensors like DHT11 or DHT22 to monitor environmental conditions and display them on an LCD.
- **Basic Robotics:** Controlling motors and servos to create simple robots or automation systems.
- **Smart Home Devices:** Integrating with Wi-Fi modules (like the ESP8266) to create remote-controlled smart home devices.
- **Game Controllers:** Building custom game controllers or input devices using the GPIO pins.

Getting Started:

- **Setting Up:** You can start by downloading and installing MicroPython or using the C/C++ SDK provided by Raspberry Pi.
- **Programming:** Use the Thonny IDE for MicroPython, which is beginner-friendly, or the Arduino IDE for C/C++ development.
- **Connecting Components:** Use a breadboard and jumper wires to connect sensors, motors, and other peripherals to the Pico's GPIO pins.
- **Powering the Pico:** You can power it via a micro-USB cable or an external power source connected to the VSYS pin.

1. *Micro Servo Motor*

A **micro servo motor** is a compact and lightweight servo motor designed for applications where space and weight are limited. Despite its small size, it provides precise control over angular position, making it ideal for robotics, RC (radio-controlled) vehicles, and other small-scale automation projects.

Key Features of a Micro Servo Motor:

1. Size and Weight:

- Compact Design: Typically, micro servo motors are very small, often about 20mm in length and weighing around 9g to 12g.

- Lightweight: Their low weight makes them suitable for applications where weight is a critical factor, such as in small drones or RC planes.

2. Control:

- Pulse Width Modulation (PWM): Micro servo motors are controlled using PWM signals. The width of the PWM pulse determines the angle of the servo arm.

- Angular Range: Most micro servos can rotate approximately 180 degrees (from 0° to 180°). Some specialized servos may offer a different range.

3. Power:

- Operating Voltage: Micro servos typically operate at a voltage range of 4.8V to 6V, which makes them compatible with standard AA batteries, LiPo batteries, or power supplies.

- Current Draw: These servos draw relatively low current, making them ideal for battery-powered applications.

4. Torque:

- Torque Rating: Micro servos generally provide torque in the range of 1.5 kg·cm to 2.5 kg·cm, depending on the model. This is sufficient for light-duty tasks like controlling small levers or flaps.

- Speed: The rotation speed of micro servos is typically around 0.1 to 0.2 seconds per 60 degrees of rotation at 6V.

5. Construction

- Plastic Gears: Most micro servos use plastic gears to keep them lightweight and affordable. However, some higher-end models may use metal gears for added durability.

- Servo Horns: They come with various types of servo horns (arms) that attach to the output shaft, allowing you to connect the servo to your project.

Applications of Micro Servo Motors:

- **Robotics:** Used to control joints, grippers, or other moving parts in small robots.

- **RC Vehicles:** Commonly used in RC cars, planes, and boats to control steering, flaps, and other mechanisms.

- **Automation Projects:** Ideal for small-scale automation tasks, such as controlling valves, switches, or other mechanical devices.

- **DIY Projects:** Frequently used in hobby projects for tasks like rotating a camera mount, operating a small door, or moving model parts.

Considerations:

- **Power Supply:** Ensure your power source can provide enough current for the servo, especially if you are controlling multiple servos simultaneously.

- **Load:** Avoid overloading the servo motor, as micro servos have limited torque and can be damaged if forced to move beyond their capacity.

Example Usage with Raspberry Pi Pico:

To control a micro servo motor using a Raspberry Pi Pico, you can use PWM signals. Here's a basic example using MicroPython:

How It Works:

- The `PWM` class controls the servo by generating a PWM signal on a GPIO pin.

- The `set_servo_angle()` function converts the desired angle (0 to 180 degrees) to the corresponding duty cycle for the PWM signal.

- The servo motor moves to the specified angle, and you can control it by changing the `angle` parameter in the function.

Example Usage with Raspberry Pi Pico:

To control a micro servo motor using a Raspberry Pi Pico, you can use PWM signals. Here's a basic example using MicroPython:

```
from machine import Pin, PWM
import time

# Setup PWM on the GPIO pin connected to the servo
servo = PWM(Pin(15))
servo.freq(50)  # Set frequency to 50Hz, typical for servos

def set_servo_angle(angle):
    # Convert the angle (0-180) to a duty cycle (between ~1000 and 9000)
    duty = int(1000 + (angle / 180.0) * 8000)
    servo.duty_u16(duty)

# Example usage
while True:
    set_servo_angle(0)   # Move servo to 0 degrees
    time.sleep(1)
    set_servo_angle(90)  # Move servo to 90 degrees (center)
    time.sleep(1)
    set_servo_angle(180) # Move servo to 180 degrees
    time.sleep(1)
```

Pin names

Name	Description
PWM	Servo control signal
V+	Positive voltage (5V)
GND	Ground

1. *Passive Infrared (PIR) sensor*

A **PIR (Passive Infrared) sensor** is an electronic sensor that measures infrared (IR) light radiating from objects in its field of view. Here's an overview of the PIR sensor and its role in a smart door lock system:

- **Functionality:**
  - PIR sensors detect motion by sensing changes in infrared radiation levels, which are emitted by warm objects, like human bodies.
  - They are called "passive" because they do not emit any energy themselves; instead, they detect the IR energy naturally emitted by objects.
- **Working Principle:**
  - The sensor consists of two slots made of a material that is sensitive to IR radiation. When the sensor is inactive, both slots detect the same amount of IR radiation. When a warm object, such as a human, moves across the sensor's field of view, it causes a change in IR radiation levels between the two slots, triggering the sensor.

Pin names

Name	Description
GND	Ground
OUT	Output (digital)
VCC	Supply voltage

2. Components of a PIR Sensor:

- **Pyroelectric Sensor:** The core component that detects infrared radiation.
- **Fresnel Lens:** Focuses the infrared signals onto the pyroelectric sensor, enhancing sensitivity and range.
- **Amplifier:** Amplifies the small signals detected by the pyroelectric sensor.
- **Comparator:** Processes the signal and triggers an output when motion is detected.

3. Applications of PIR Sensors:

- **Security Systems:** Widely used in security systems to detect intruders by sensing motion.
- **Automated Lighting:** Used in automatic lighting systems that turn on lights when motion is detected.
- **Home Automation:** Integrated into smart home devices to trigger actions, such as locking/unlocking doors or sending alerts.

4. PIR Sensor in a Smart Door Lock System:

- **Motion Detection:**
  - The PIR sensor can be used to detect when someone approaches the door. This information can be used to trigger the lock mechanism or alert the homeowner via an app.

- **Enhanced Security:**
  - Integrating a PIR sensor with a smart door lock adds an extra layer of security by detecting unauthorized motion near the door and taking appropriate actions, such as locking the door or sending an alert.
- **Automation:**
  - In combination with other sensors (like an LDR for light detection), a PIR sensor can automate the door lock system based on both motion and environmental conditions.

5. Advantages of Using PIR Sensors:

- **Low Power Consumption:** PIR sensors consume very little power, making them suitable for battery-operated systems.
- **Wide Range and Sensitivity:** They can detect motion over a wide area, making them effective for security purposes.
- **Cost-Effective:** PIR sensors are relatively inexpensive and widely available.

6. Integration in IoT Systems:

- **Data Communication:** The PIR sensor can communicate its data to the main controller (like a Raspberry Pi Pico) in the smart door lock system. This data can be processed to decide whether to lock/unlock the door or trigger an alert.
- **Wireless Connectivity:** In IoT-based systems, the sensor data can be sent to the cloud, allowing users to monitor their home remotely and receive real-time alerts.

Example Code Integration:

```
from machine import Pin

import time


# Initialize the PIR sensor

pir_sensor = Pin(14, Pin.IN)  # PIR sensor connected to GPIO14 (can be changed as per your wiring)


# Function to simulate unlocking the door

def unlock_door():

  print("Door Unlocked!")  # Replace this with actual code to control your servo motor or lock mechanism
```

```
# Function to simulate sending an alert

def send_alert():

  print("Motion detected! Alert sent to user!")  # Replace this with actual code to send a notification


# Main loop to monitor the PIR sensor

while True:

  if pir_sensor.value() == 1:  # PIR sensor output is HIGH when motion is detected

    print("Motion detected!")

    unlock_door()  # Call the function to unlock the door

    send_alert()  # Call the function to send an alert

    time.sleep(10)  # Wait for 10 seconds to avoid multiple triggers


else:

    print("No motion detected.")

  time.sleep(1)  # Check the PIR sensor every second
```

Explanation:

1. **PIR Sensor Initialization:**
   - The pir_sensor is initialized on GPIO14 of the Raspberry Pi Pico. You can connect the PIR sensor's output pin to any GPIO pin and adjust the code accordingly.
2. **Main Loop:**
   - The main loop continuously checks the value of the PIR sensor.
   - If motion is detected (pir_sensor.value() == 1), it triggers two functions: unlock_door() and send_alert().
3. **Unlock Door:**
   - The unlock_door() function simulates unlocking the door. In a real application, you would replace the print ("Door Unlocked!") line with code that controls a servo motor or an electronic lock.
4. **Send Alert:**
   - The send alert () function simulates sending an alert to the user. This could be implemented using an IoT service like Blynk or sending an email or SMS.
5. **Debounce Logic:**
   - After detecting motion, the code waits for 10 seconds before it checks again. This prevents multiple triggers from the same motion event.

6. **No Motion:**
   - If no motion is detected, the code simply prints "No motion detected." and continues to monitor the sensor.

1. *USB Type-B Cable*

In a smart door lock system, the **USB Type-B cable** could play a role in powering certain components or providing a connection for programming and data transfer. Here's how it might be utilized:

1. Power Supply:

- **Peripheral Devices:** If your smart door lock system includes components that require a stable power supply, such as certain types of controllers or sensors, a USB Type-B cable could be used to connect these devices to a power source, such as a computer or a dedicated USB power adapter.
- **External Modules:** For example, if the system uses a controller that has a USB Type-B port (like some older development boards or modules), the cable would provide the necessary power to operate the system.

2. Data Communication:

- **Programming the Microcontroller:** If the microcontroller or development board in your smart door lock system has a USB Type-B port, the cable can be used to connect it to a computer for programming. This allows you to upload code, such as the control logic for the door lock, directly to the microcontroller.
- **Firmware Updates:** The USB Type-B cable could also be used for firmware updates, allowing you to easily update the system's software if needed.

3. Connection to External Devices:

- **Debugging and Testing:** During the development and testing phase of your smart door lock system, a USB Type-B cable might be used to connect the microcontroller to a computer for debugging purposes. This allows you to monitor the system's operation in real-time and make adjustments as necessary.

4. Legacy Devices:

- **Older Modules or Components:** If your smart door lock system incorporates older hardware that uses a USB Type-B interface, the cable would be essential for connecting these devices to other components or power sources within the system.

1. *Connecting Wires*

• *Female to Male or Male to Female*

Connecting **female-to-male wires** (often referred to as **jumper wires**) is common in electronics projects for making connections between different components, such as microcontrollers, sensors, and actuators.

In a smart door lock system, using **female-to-male jumper wires** is common for making connections between components, such as between a microcontroller (like the Raspberry Pi Pico) and other devices (such as a servo motor or sensors).

Understanding Female-to-Male Jumper Wires:

- **Female Connector:** The female end of the jumper wire has a socket that fits over the pins of components or headers on a breadboard. It is useful for connecting to the male pins of a component or breadboard.
- **Male Connector:** The male end of the jumper wire has a pin that fits into the female sockets of components, headers, or breadboards.

Common Use Cases in a Smart Door Lock System:

1. **Connecting the Microcontroller:**
   - **Microcontroller to Servo:** Connect the control pin of the servo motor to a GPIO pin on the microcontroller.
2. **Connecting Sensors:**
   - **PIR to ADC Pin:** Connect the PIR sensor to an ADC pin on the microcontroller. You'll use female-to-male wires to connect the LDR's output pin to the ADC input pin on the Pico.
   - **Other Sensors:** If using additional sensors (like a distance sensor or a temperature sensor), you'll similarly use female-to-male wires to connect their output to the appropriate pins on the microcontroller.
3. **Power Connections:**
   - **Powering Components:** Female-to-male wires can also be used to connect the power and ground pins of the components to the power rails on a breadboard or directly to the microcontroller's power and ground pins.

- ## *Female to Female Wires*

Female-to-female wires (often called **female-to-female jumper wires**) are used to make connections between two components or devices that both have male pins or headers. They are versatile and helpful in a variety of electronics and prototyping tasks.

Female-to-female jumper wires are used in electronics to make connections between components with male pins or headers. In a smart door lock system, these wires are especially useful for connecting various parts of your circuit on a breadboard or directly between components.

Applications of Female-to-Female Jumper Wires:

1. **Connecting Components to a Breadboard:**
   - **Breadboard Connections:** If you have components with male headers (such as sensors or microcontrollers), you can use female-to-female wires to connect these components to the breadboard. The wires plug into the female sockets on the breadboard, allowing easy routing of connections.
2. **Interfacing Sensors and Actuators:**
   - **Sensors to Microcontroller:** Connect sensors (like a photoresistor or door contact sensor) that have male pins to the corresponding input pins on your microcontroller using female-to-female wires.
   - **Actuators to Power Supply:** If your smart lock system includes actuators like servos, you might use female-to-female wires to connect these actuators to the power supply or control pins.
3. **Connecting Modules and Accessories:**
   - **Peripheral Modules:** Use female-to-female wires to connect peripheral modules (e.g., Bluetooth modules, relay boards) to the microcontroller or other control circuits.
4. **Power Distribution:**
   - **Power and Ground:** Connect power and ground rails on a breadboard to various components in your smart lock system using female-to-female wires. This is particularly useful for distributing power and ground connections across multiple components.

1. *Cardboard and Glue*

Using **cardboard** and **glue** in a smart door lock system can be an effective and cost-efficient way to prototype and assemble the physical components of the system.

Cardboard can be used for:

- **Enclosures:** Create a simple enclosure or mounting platform for your electronic components, such as the microcontroller, sensors, and servo motor. This helps protect the components and keeps them organized.
- **Mounting Brackets:** Construct brackets or mounts to hold the servo motor and other components in place, ensuring they are securely positioned to interact with the locking

mechanism.

Glue can be used for:

- **Assembly:** Securely attach components to the cardboard, such as gluing the microcontroller to the base of the enclosure or attaching the servo motor to a bracket.
- **Reinforcement:** Strengthen joints or connections between different parts of your cardboard structure to ensure stability and durability.

Code

```
import network

import socket

from time import sleep

from machine import Pin,PWM

from picozero import Servo
```

```
servo = Servo(5)
// Your WiFi credentials.
// Set password to "" for open networks.
ssid = 'POCO M6 Pro 5G'
password = 'Anamika@1998'

pir= Pin(6,Pin.IN)

def connect():
 #Connect to WLAN
 wlan = network.WLAN(network.STA_IF)
 wlan.active(True)
 wlan.connect(ssid, password)
 while wlan.isconnected() == False:
  print('Waiting for connection...')
  sleep(1)
 ip = wlan.ifconfig()[0]
 print(f'Connected on {ip}')
 return ip

def open_socket(ip):
 # Open a socket
 address = (ip, 80)
 connection = socket.socket()
 connection.bind(address)
```

```
connection.listen(1)
 return connection

def webpage(state):
 #Template HTML
 html = f"""
   <!DOCTYPE html>
   <html>
   <form action="./DoorOpen">
   <input type="submit" value="Door Open" />
   </form>
   <form action="./DoorClose">
   <input type="submit" value="Door Close" />
   </form>
   </body>
   </html>
   """
 return str(html)

def serve(connection):
 #Start a web server
 state = 'Close'
 servo.min()
 while True:
  pir==1
  client = connection.accept()[0]
  request = client.recv(1024)
```

```
    request = str(request)
    try:
      request = request.split()[1]
    except IndexError:
      pass
    if request == '/DoorOpen?':



        servo.max()
        state = 'Open'
  elif request =='/DoorClose?':
        servo.min()
        state = 'Close'
    #temperature = pico_temp_sensor.temp
    html = webpage(state)
    client.send(html)
    client.close()
try:
  ip = connect()
  connection = open_socket(ip)
  serve(connection)
except KeyboardInterrupt:
  machine.reset()
```

The following code needs to entered/typed in Thonny

Steps after entering the code in Thonny

- Either the piczero.py library is installed from the Thonny
- Or we are provided with the library
- The setting in the bottom right corner is to be changed to ***raspberry pico micropython***

The code can be run in wokwi website : New MicroPython on Raspberry Pi Pico Project – Wokwi Simulator

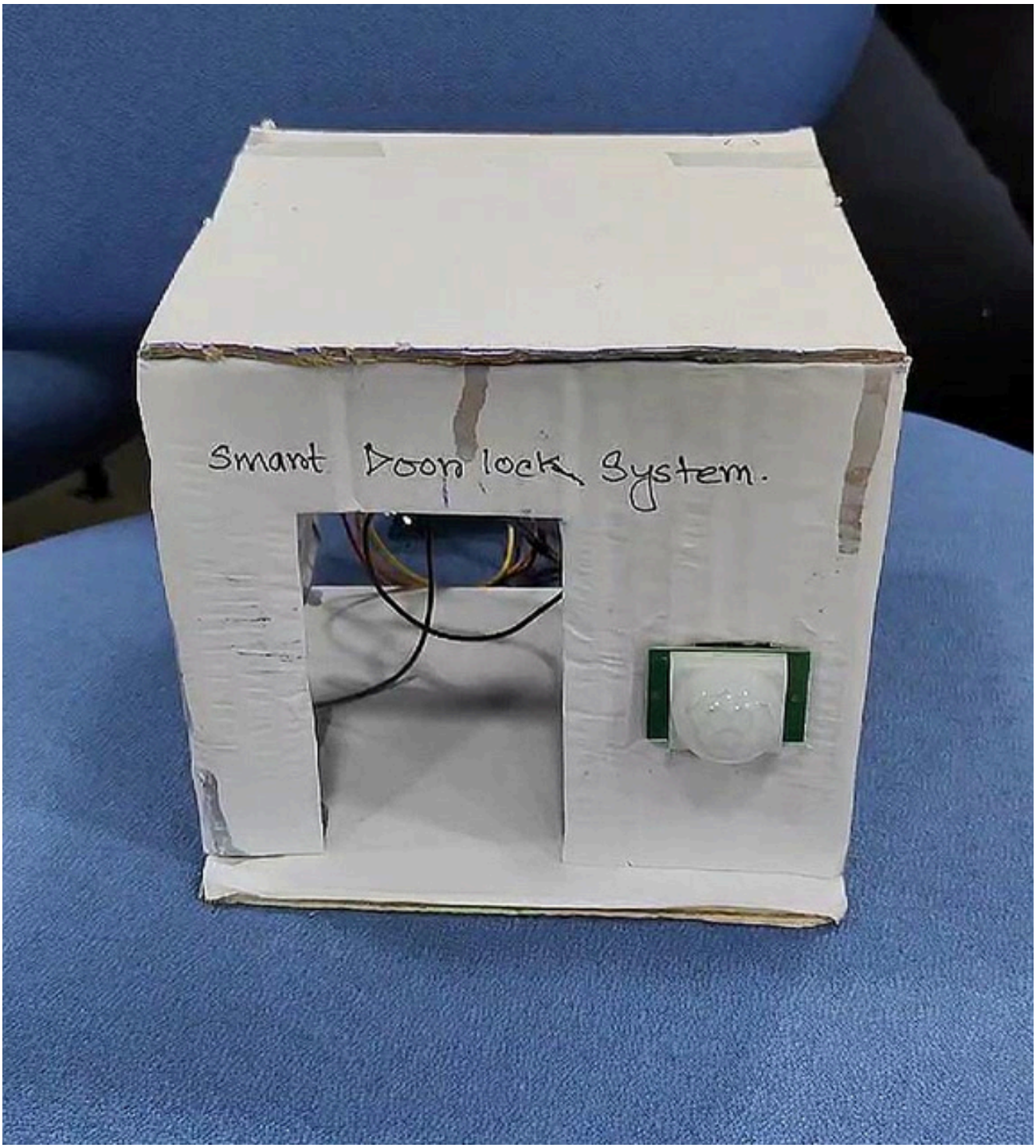
Output

As the html code is entered in the main code, it takes the user to a different web page. The number id displayed on the output window of Thonny is to pasted in the chrome or any other window.

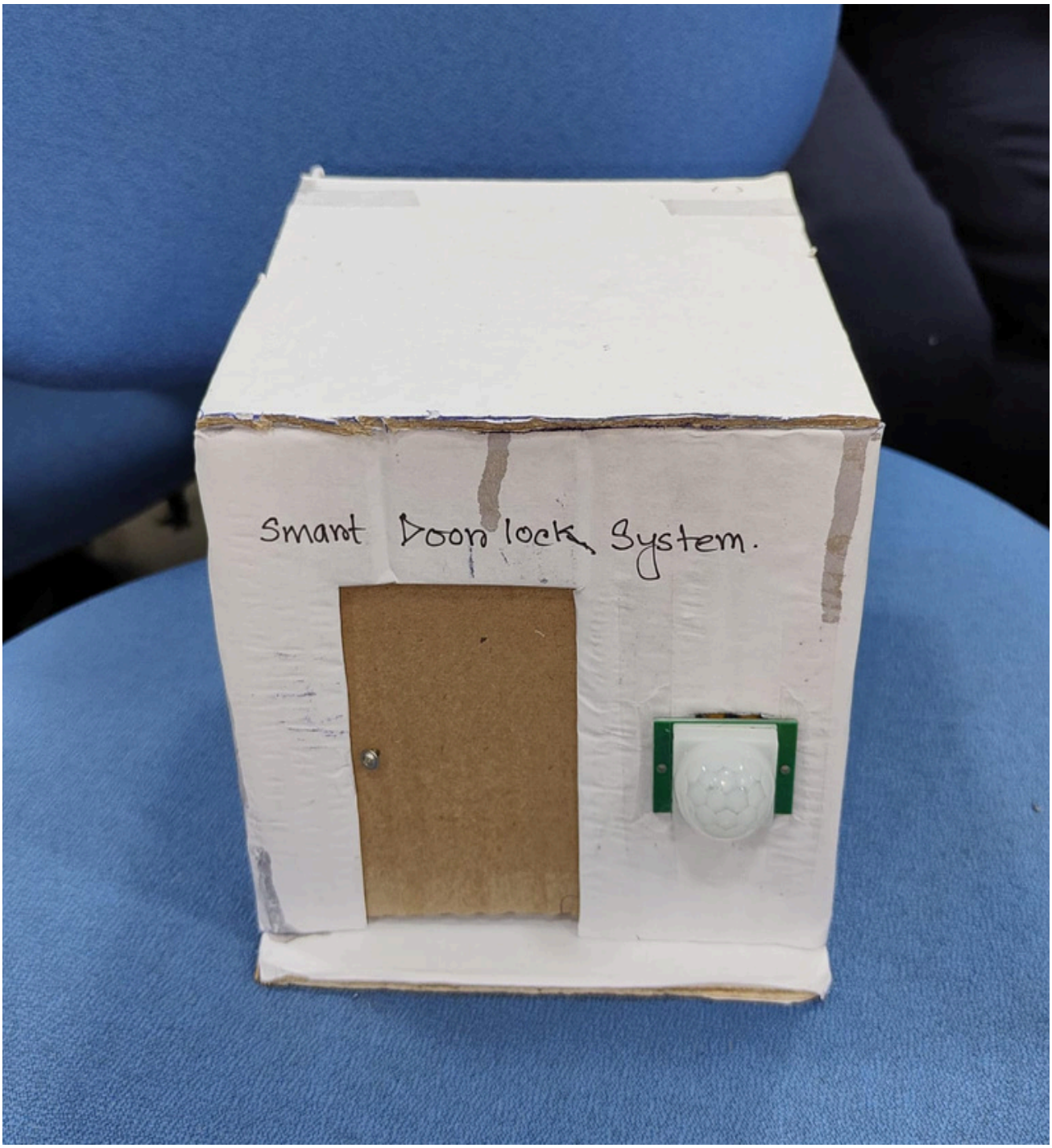
Once the above page is displayed

- The user clicks on the button “Door Open” for the door to open
- The user clicks on the button “Door Close” for the door to Close

- **DOOR OPEN**



- **DOOR CLOSE**



Detailed Explanation of Smart Door Lock System using Wifi(Local IOT)

Creating a smart door lock system using **Thonny (a Python IDE)**, **Raspberry Pi Pico**, a **servo motor**, an **PIR sensor** (passive infrared), **USB Type-B cable**, and **connecting wires** is a great project to explore IoT and embedded systems. Below is a guide to help you set up and code the system.

1. Components Overview:

- **Raspberry Pi Pico:** The microcontroller that will control the smart door lock system.
- **Servo Motor:** Controls the locking and unlocking mechanism by rotating between two positions.
- **PIR Sensor:** The PIR sensor detects motion by sensing infrared radiation from objects within its field of view, particularly the human body.
- **Micro USB Cable:** Used to power the Raspberry Pi Pico and upload the code from your computer.
- **Connecting Wires:** Female-to-female or female-to-male wires to connect the components to the Pico.

2. Wiring the Components:

- **Servo Motor:**
  - **Signal Pin (Orange/Yellow):** Connect to GP15 (or any available GPIO pin) on the Pico.
  - **Power Pin (Red):** Connect to the 3.3V or 5V pin on the Pico.
  - **Ground Pin (Brown/Black):** Connect to one of the GND pins on the Pico.
- **LDR Sensor:**
  - **One end of the LDR:** Connect to GP26 (ADC0) on the Pico.
  - **Other end of the LDR:** Connect to 3.3V on the Pico.
  - **10kΩ Resistor:** Connect one end of the resistor between the LDR's output and GND.

- **Power Connection:**
  - Use the **USB Type-B cable** to connect the Raspberry Pi Pico to your computer for power and programming.

3. Installing Thonny and Setting Up the Pico:

- **Download Thonny:** If you haven't already, download and install Thonny from thonny.org.
- **Select Interpreter:** In Thonny, go to Tools -> Options -> Interpreter. Select "MicroPython (Raspberry Pi Pico)" from the dropdown.
- **Connect the Pico:** Plug in the Raspberry Pi Pico via the USB Type-B cable.

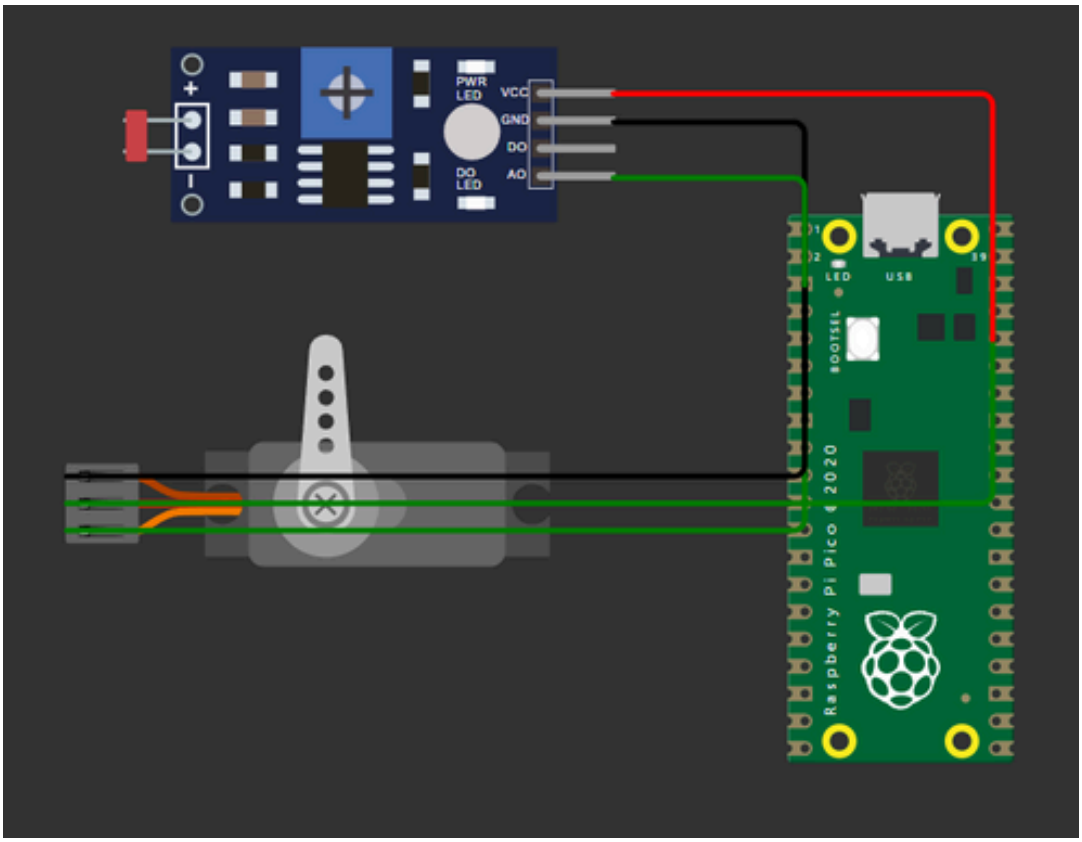
4. Uploading and Running the Code:

- **Write the Code:** Write the code in Thonny.
- **Access the Web Surface:** Once the Raspberry Pi Pico is connected to Wi-Fi and the server is running, open a web browser and enter the Pico's IP address (which will be printed in the console).You can lock or unlock the door by navigating to /lock or /unlock in the browser.
- **Save the Code:** Save the code to the Raspberry Pi Pico as main.py so it runs automatically on boot.
- **Run the Code:** Click the "Run" button in Thonny to upload and execute the code on the Pico.
- **Monitor and Control:** The LDR sensor will continuously monitor the light level and adjust the lock position accordingly. You can also control the lock manually through the web interface.

5. Testing the System:

- **Light Conditions:** Test the system by changing the light conditions around the LDR. The servo should move between the lock (0 degrees) and unlock (90 degrees) positions based on the light level.
- **Adjustments:** You may need to adjust the threshold value for the LDR or fine-tune the servo angles depending on your specific lock mechanism.

Circuit


PWR LED
VCC
GND
DO
AO
DO LED
LED
USB
BOOTSEL
Raspberry Pi Pico © 2020

Future Scope

The future scope of a Smart Door Lock System is vast, driven by advancements in technology, growing concerns about security, and the increasing adoption of smart home systems. Here are some potential areas where smart door lock systems could evolve:

1. Enhanced Security Features:

- **Biometric Integration:** Future systems may incorporate advanced biometric sensors such as facial recognition, voice recognition, or even behavioural biometrics (e.g., gait analysis) for more secure and personalized access control.
- **Multi-Factor Authentication:** Combining different authentication methods, such as biometrics, PIN codes, and mobile app verification, will enhance security, making unauthorized access extremely difficult.
- **AI-Powered Threat Detection:** AI algorithms could analyse patterns of lock usage and detect unusual behaviour, automatically alerting homeowners or authorities in case of suspicious activity.

2. Cloud-Based Management and Analytics:

- **Remote Management:** Cloud-based platforms will allow users to manage and monitor their smart door locks from anywhere in the world. This includes generating temporary access codes, viewing access logs, and receiving real-time alerts.
- **Data Analytics:** Analysis of usage data could provide insights into patterns, helping to optimize access schedules, predict maintenance needs, or even suggest energy-saving measures by coordinating with other smart devices.

3. Improved User Experience:

- **Seamless Entry:** Future smart locks might eliminate the need for physical keys, PIN codes, or even smartphones, allowing users to enter their homes through invisible, automated systems that recognize their presence (e.g., through wearables or mobile devices).
- **Customizable Access Control:** Users could easily customize access rights for different people or groups, such as family members, guests, or service providers, with time-based or location-based restrictions.

4. Wider Market Adoption:

- **Cost Reduction:** As technology matures, the cost of smart door lock systems is expected to decrease, making them more accessible to a broader range of consumers.
- **Standardization:** Industry standards for smart lock technology could emerge, ensuring compatibility and interoperability across different brands and platforms, further driving adoption.

5. Integration with Public and Commercial Spaces:

- **Smart Cities:** In smart cities, smart locks could be integrated into public infrastructure, such as shared access buildings, coworking spaces, or rental properties, offering secure, easy-to-manage access control.
- **Commercial and Industrial Applications:** Beyond residential use, smart locks will increasingly be adopted in commercial and industrial settings, where they can provide secure access control, track employee movements, and integrate with broader security and operational systems.

Conclusion

The **Smart Door Lock System** represents a significant advancement in home security and automation, offering a blend of convenience, enhanced safety, and modern technology. By integrating components such as a **Raspberry Pi Pico**, **servo motor**, **PIR sensor**, and utilizing platforms like **Blynk IoT**, users can remotely control and monitor access to their homes, providing both manual and automated control over entry points.

This system not only increases the security of residential and commercial properties but also offers the potential for future enhancements such as biometric authentication, AI-driven security analytics, and seamless integration with other smart home devices. As smart technology continues to evolve, smart door locks will likely become an essential feature of modern homes, contributing to the development of smarter, safer living environments.

Ultimately, the Smart Door Lock System exemplifies the intersection of IoT, convenience, and security, paving the way for more intelligent and responsive home automation solutions.